

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization.

Decision Tree Overfitting: Causes, Debugging, and Optimization (10 Marks)

Introduction

A **Decision Tree** is a supervised machine learning algorithm used for classification and regression. It splits data into branches based on feature values to make predictions. If a decision tree achieves **high accuracy on training data but poor performance on unseen (test) data**, it indicates that the model is **overfitting** and has poor **generalization**.

1. Possible Causes of Poor Performance

(a) Overfitting

Overfitting occurs when the decision tree learns the training data too closely, including noise and outliers, instead of general patterns.

Effects:

- Very high training accuracy.
- Low test accuracy.
- Poor prediction on new data.

(b) Deep Tree Structure

A tree with too many levels becomes highly complex.

Problems:

- Memorizes training samples.
- Creates unnecessary branches.
- Increases variance.

(c) Noisy Data

Noisy or incorrect data includes:

- Missing values
- Incorrect labels
- Outliers
- Measurement errors

The tree may create splits based on this noise, leading to inaccurate predictions.

2. Debugging Steps

To identify and fix the problem, the following steps can be performed:

Step 1: Compare Training and Testing Accuracy

- If training accuracy is much higher than testing accuracy, the model is likely overfitting.

Step 2: Visualize the Tree

Examine the tree structure.

- Check the depth of the tree.
- Identify unnecessary branches.
- Look for leaves containing very few samples.

3. Optimization Methods

(a) Pruning

Pruning reduces the size of the decision tree by removing unnecessary branches.

Types of Pruning

1. Pre-Pruning (Early Stopping)

Stops tree growth before it becomes too deep.

Common parameters:

- Maximum depth (max_depth)
- Minimum samples per split (min_samples_split)
- Minimum samples per leaf (min_samples_leaf)

Advantages:

- Prevents overfitting.
- Faster training.
- Simpler model.

2. Post-Pruning

The tree is first grown fully and then less useful branches are removed.

Advantages:

- Improves generalization.
 - Reduces model complexity.
 - Often provides better accuracy on unseen data.
-

(b) Cross-Validation

Cross-validation evaluates the model on multiple subsets of the data to ensure it performs well on unseen samples.

A common approach is **k-Fold Cross-Validation**:

- Split the dataset into k equal parts.
- Train on $k - 1$ parts.

Advantages:

- Detects overfitting.
 - Provides a reliable estimate of model accuracy.
 - Helps in selecting the best model.
-

(c) Feature Selection

Feature selection removes irrelevant or redundant attributes.

Common techniques:

- Information Gain
- Chi-Square Test
- Correlation Analysis
- Recursive Feature Elimination (RFE)

Benefits:

- Reduces model complexity.
- Improves prediction accuracy.
- Faster training.
- Better generalization.

4. Benefits of Optimization

After applying pruning, cross-validation, and feature selection:

- Better generalization on unseen data.
- Reduced overfitting.
- Improved prediction accuracy.
- Simpler and more interpretable model.
- Faster training and prediction.

5. Conclusion

A decision tree that performs exceptionally well on training data but poorly on unseen data is usually suffering from **overfitting**, often caused by an overly deep tree, noisy data, or irrelevant features. The problem can be addressed by analyzing the model, cleaning the data, and applying optimization techniques such as **pruning**, **cross-validation**, **feature selection**, and **hyperparameter tuning**. These methods reduce model complexity, improve generalization, and produce a more reliable decision tree that performs well on both training and unseen datasets.

6. A convolutional neural network (CNN) used for image classification produces unstable results across training epochs. Examine potential issues such as improper data augmentation, vanishing gradients, or batch normalization errors. Describe debugging strategies and suggest optimization techniques to stabilize training and improve accuracy

A **Convolutional Neural Network (CNN)** may produce unstable results across training epochs due to problems such as improper data augmentation, vanishing gradients, or batch normalization errors. These issues can reduce accuracy and slow learning.

Potential Issues

1. Improper Data Augmentation

- Excessive rotation, cropping, or flipping changes important image features.
- Leads to inconsistent training and poor generalization.

2. Vanishing Gradients

- Gradients become very small in deep networks.
- Earlier layers learn very slowly, causing unstable or poor convergence.

3. Batch Normalization Errors

- Incorrect placement or small batch sizes produce inaccurate statistics.
- Results in unstable training and fluctuating validation accuracy.

Debugging Strategies

- Visualize augmented images to ensure transformations are reasonable.
- Monitor **training and validation loss/accuracy** to detect overfitting or underfitting.
- Check gradient values to identify vanishing or exploding gradients.
- Verify the correct implementation of batch normalization and learning rate.
- Train on a small subset of data to confirm the model can learn correctly.

Optimization Techniques

- Apply **balanced data augmentation** to improve generalization.
- Use **ReLU activation** and **He/Xavier initialization** to reduce vanishing gradients.
- Add **Batch Normalization** after convolution layers for stable learning.
- Use an **adaptive optimizer** such as **Adam** and tune the learning rate.
- Apply **Dropout**, **Early Stopping**, and **Learning Rate Scheduling** to improve accuracy and prevent overfitting.

Conclusion

By identifying issues in data augmentation, gradients, and batch normalization, and applying proper debugging and optimization techniques, CNN training becomes more stable, converges faster, and achieves higher classification accuracy.